DESIGN AND IMPLEMENTATION OF 32 BIT RISC-V PROCESSOR WITH PARALLEL GRAPHICS ACCELERATOR

A MAJOR PROJECT PHASE II REPORT

Submitted in partial fulfillment of the requirement for the award of the degree

Bachelor of Engineering

In

DEPARTMENT OF ELECTRONICS (VLSI DESIGN AND TECHNOLOGY)

Submitted By

NANDAVARDHAN S G	**1MJ23VL020**
ROHITH SHEKAR	**1MJ23VL024**
SUMUKH HEGDE	**1MJ23VL026**
YASHAVANTH K	**1MJ23VL030**

Under the Guidance of

Dr. MANISHA

Assistant Professor

Department of Electronics Engineering (VLSI Design and Technology), MVJCE

MVJ College of Engineering, Bengaluru

(An Autonomous Institute)

Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi

Recognized by UGC with 12(f) & 12(b), Accredited by NBA & NAAC

MAY 2026

MVJ COLLEGE OF ENGINEERING, BENGALURU-560067

(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF ELECTRONICS (VLSI DESIGN AND TECHNOLOGY) ENGINEERING

CERTIFICATE

Certified that the Major Project Phase II work titled **"DESIGN AND IMPLEMENTATION OF 32 BIT RISC-V PROCESSOR WITH PARALLEL GRAPHICS ACCELERATOR"** is carried out by **NANDAVARDHAN S G (1MJ23VL020), ROHITH SHEKAR (1MJ23VL024), SUMUKH HEGDE (1MJ23VL026), YASHAVANTH K (1MJ23VL030)**, who are Bonafide students of MVJ College of Engineering, Bengaluru, in partial fulfillment for the award of degree of **Bachelor of Engineering in Electronics Engineering (VLSI Design and Technology)** of the Visvesvaraya Technological University, Belagavi during the year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the major project phase II report deposited in the departmental library. The major project phase II report has been approved as it satisfies the academic requirements in respect of major project work prescribed by the institution for the said Degree.

Signature of Guide	**Signature of Head of the Department**	**Signature of Dean**
Dr. MANISHA Assistant Professor Dept. of VLSI MVJ College of Engineering Bengaluru	**Dr. REMASHAN KARIYAADAN** Head of the Department Dept. of VLSI MVJ College of Engineering Bengaluru	**Dr. M. Brindha** Dean School of Electrical Science MVJ College of Engineering Bengaluru

External Viva

Name of Examiners	**Signature with Date**
1	
2	

MVJ COLLEGE OF ENGINEERING, BENGALURU-560067
(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF ELECTRONICS ENGINEERING (VLSI DESIGN AND TECHNOLOGY)

DECLARATION

We **NANDAVARDHAN S G (1MJ23VL020), ROHITH SHEKAR (1MJ23VL024), SUMUKH HEGDE (1MJ23VL026), YASHAVANTH K (1MJ23VL030)**, students of sixth semester B.E., Department of Electronics (VLSI Design And Technology), MVJ College of Engineering, Bengaluru, hereby declare that the major project phase II titled **'DESIGN AND IMPLEMENTATION OF 32 BIT RISC-V PROCESSOR WITH PARALLEL GRAPHICS ACCELERATOR'** has been carried out by us and submitted in partial fulfilment for the award of Degree of Bachelor of Engineering in Electronics (VLSI Design And Technology) during the year 2025-2026.

Further we declare that the content of the dissertation has not been submitted previously by anybody for the award of any Degree or Diploma to any other University.

We also declare that any Intellectual Property Rights generated out of this project carried out at MVJCE will be the property of MVJ College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru Date:

Name **Signature**

1. **NANDAVARDHAN S G (1MJ23VL020)**
2. **ROHITH SHEKAR (1MJ23VL024)**
3. **SUMUKH HEGDE (1MJ23VL026)**
4. **YASHAVANTH K (1MJ23VL030)**

ABSTRACT

Graphics generation is a data-parallel task that a general-purpose scalar processor performs inefficiently, since every pixel requires several instructions. This project presents the design and implementation of a 32-bit RISC-V processor based system with a parallel graphics accelerator on the Nexys A7-100T FPGA (Xilinx Artix-7 XC7A100T). Phase I established the literature background, architecture and requirements; Phase II, reported here, implements the system in Verilog and validates it in simulation and on hardware.

The design consists of an instruction ROM and RISC-V processor interface, a memory-mapped GPU register block located at address 0x4000_0000, a parameterised parallel graphics accelerator pixel datapath, a 320 × 240 frame buffer with 12 bits per pixel mapped onto block RAM, a divide-by-four pixel clock enable that derives 25 MHz from the 100 MHz system clock, and a 640 × 480 VGA timing controller with active-low synchronisation signals at about 59.5 Hz. The frame buffer is displayed at 2 × scale on the monitor.

The design was verified in Vivado behavioural simulation by observing the program counter, fetched instruction, VGA_HS, VGA_VS and the red, green and blue outputs, and was then programmed on the Nexys A7-100T, where a sharp centred red rectangle of 320 × 240 pixels was displayed on a VGA monitor, as predicted by the design calculations.

Finally, on-chip power was analysed with Vivado. The total on-chip power of the earlier build was 6.098 W (implemented design), while the current design is estimated at 2.675 W from the synthesised netlist, a reduction of 3.423 W (56.1 %), mainly from I/O and logic power, with the junction temperature falling from 52.8 °C to 37.2 °C. The results show that a compact RISC-V and accelerator based display system can be realised with a small resource and power budget.

ACKNOWLEDGEMENT

We express our sincere gratitude to our beloved Principal, **Dr. Ajayan K R** for all his support towards this project work.

We express our heartfelt gratitude to **Dr. M Brindha**, Dean, School of Electrical Science, for her constant encouragement, valuable guidance, and unwavering support throughout the course of our project work.

Our sincere thanks to **Dr. REMASHAN KARIYAADAN**, Head of the Department, Electronics Engineering (VlSI Design And Technology), MVJCE for his support and encouragement.

We are indebted to our guide, **Dr. MANISHA**, Assistant Professor, Dept of Electronics Engineering (VlSI Design And Technology), MVJ College of Engineering for her wholehearted support, suggestions, and invaluable advice throughout our project work and for the help in the preparation of this report.

We thank to all the technical and non-technical staff of Electronics Engineering (VlSI Design And Technology) Department, MVJCE for their help.

TABLE OF CONTENT

Abstract..... III
Acknowledgement..... IV
Table of Content..... V
Table of Figures..... VII
Table of Tables..... VIII
CHAPTER 1 INTRODUCTION..... 1
CHAPTER 2 LITERATURE SURVEY..... 3
2.1 The RISC-V Instruction Set Manual, Volume I: User-Level ISA..... 3
2.2 Instruction Sets Should Be Free: The Case for RISC-V..... 3
2.3 The Rocket Chip Generator..... 3
2.4 Slow and Steady Wins the Race? A Comparison of Ultra-Low-Power RISC-V Cores for IoT Applications..... 4
2.5 Near-Threshold RISC-V Core with DSP Extensions for Scalable IoT Endpoint Devices..... 4
2.6 PicoRV32: A Size-Optimised RISC-V CPU..... 5
2.7 Vortex: Extending the RISC-V ISA for GPGPU and 3D Graphics..... 5
2.8 FlexGrip: A Soft GPGPU for FPGAs..... 5
2.9 Enabling GPGPU Low-Level Hardware Explorations with MIAOW..... 6
2.10 Nyami: A Synthesizable GPU Architectural Model for General-Purpose and Graphics-Specific Workloads..... 6
2.11 GPU Computing..... 7
2.12 FPGA Prototyping by Verilog Examples..... 7
2.13 VESA Monitor Timing Standard (DMT)..... 7
2.14 Measuring the Gap Between FPGAs and ASICs..... 8
2.15 Power Estimation Techniques for FPGAs..... 8
2.16 Dynamic Power Consumption in Virtex-II FPGA Family..... 8
CHAPTER 3 PROBLEM STATEMENT..... 10
CHAPTER 4 EXISTING AND PROPOSED SYSTEM..... 11
4.1 EXISTING SYSTEM..... 11
4.1.1 DISADVANTAGES OF EXISTING SYSTEM..... 11
4.2 PROPOSED SYSTEM..... 11
4.2.1 ADVANTAGES OF PROPOSED SYSTEM..... 12
CHAPTER 5 METHODOLOGY..... 14
5.1 System Overview..... 14
5.2 Design Specification..... 14
5.3 RISC-V Processor and Instruction ROM..... 15
5.4 GPU Register Interface (MMIO)..... 15
5.5 Parallel Graphics Accelerator..... 15
5.6 Frame Buffer..... 16
5.7 VGA Timing Controller..... 16
5.8 Read-Address Generation and Display Scaling..... 17
5.9 Demonstration Writer..... 17
5.10 Simulation Setup..... 18
5.11 Constraints and Implementation..... 18
5.12 Hardware Validation..... 18
5.13 Power Analysis Method..... 19

5.14 System Outcome..... 19
CHAPTER 6 HARDWARE AND SOFTWARE REQUIREMENTS.....20
6.1 HARDWARE REQUIREMENTS.....20
6.2 SOFTWARE REQUIREMENTS.....20
CHAPTER 7 RESULTS.....21
7.1 Behavioural Simulation Results.....21
7.2 Hardware Results.....22
7.3 Power Analysis..... 26
7.4 Power Comparison and Power Reduction.....27
CONCLUSION.....28
REFERENCES.....29

LIST OF FIGURES

Fig 5.1 System Architecture and Methodology Block Diagram.....14
Fig 5.2 VGA Horizontal and Vertical Timing.....17
Fig 5.3 Frame Buffer Contents and Resulting VGA Output.....18
Fig 7.1 Processor Signals: pc, instr and status.....21
Fig 7.2 Simulation Waveform of VGA Synchronisation and Colour Outputs.....21
Fig 7.3 Simulation at 1 ms.....22
Fig 7.4 Simulation at 4 ms.....22
Fig 7.5 Hardware Set-up: Nexys A7-100T Driving the VGA Monitor.....23
Fig 7.6 Hardware Output (Second View of the Set-up).....24
Fig 7.7 Close-up of the Red Rectangle on the Monitor.....25
Fig 7.8 Power Report: Current Design (Synthesised Netlist Estimate).....26
Fig 7.9 Power Report: Earlier Build (Implemented Design).....26
Fig 7.10 Power by Category: Earlier Build versus Current Design.....27

LIST OF TABLES

Table 5.1 Design Parameters.....14
Table 5.2 GPU Register Map.....15
Table 5.3 VGA Timing Parameters (640 × 480).....16
Table 5.4 Pin Assignments Used in the Constraint File.....18
Table 7.1 Verification Summary.....22
Table 7.2 Power Comparison.....27

CHAPTER 1

INTRODUCTION

Visual output has become a basic requirement of modern embedded and edge systems, from instrument panels and industrial terminals to teaching platforms and gaming devices. Graphics workloads such as clearing a frame, filling a region or updating a frame buffer are highly data-parallel: the same simple operation is repeated for thousands of pixels. When such work is performed by a general-purpose scalar processor, every pixel costs several instructions, which wastes processor time and energy. Pairing a processor with a dedicated graphics accelerator moves this repetitive work into hardware and leaves the processor free for control tasks.

RISC-V is an open and royalty-free instruction set architecture (ISA) with a small, regular base integer instruction set and modular extensions. Its simplicity makes it well suited to academic System-on-Chip (SoC) design, FPGA prototyping and research on custom accelerators, because a designer can implement the base ISA without licensing constraints and add memory-mapped peripherals around it.

Field Programmable Gate Arrays (FPGAs) allow such designs to be validated on real hardware before any silicon commitment. This project targets the Digilent Nexys A7-100T board, which carries a Xilinx Artix-7 XC7A100T device with 63,400 look-up tables, 135 block RAM tiles (4,860 Kb in total), 240 DSP48E1 slices, a 100 MHz oscillator and a 12-bit (4 bits per colour) VGA output. These resources are sufficient to hold a complete frame buffer on chip and drive a standard VGA monitor directly.

The project, titled **“Design and Implementation of 32 Bit RISC-V Processor with Parallel Graphics Accelerator”**, develops a 32-bit RISC-V based system in which the processor controls a graphics accelerator through a memory-mapped register interface, the accelerator writes pixels into an on-chip frame buffer, and a VGA controller continuously scans the frame buffer to a monitor at 640 × 480 and approximately 60 Hz. Phase I of the project established the literature background, the architecture and the requirements. Phase II, reported here, covers the RTL implementation in Verilog, behavioural simulation in Vivado, realisation on the Nexys A7-100T board, observation of the output on a VGA monitor, and on-chip power analysis and reduction.

The objectives of Phase II are:

- To implement the display path (frame buffer, VGA timing controller and pixel clock enable) in synthesisable Verilog.
- To implement the processor-visible GPU register interface, the pixel datapath of the graphics accelerator and an instruction ROM for the processor.

- To verify the design by behavioural simulation, observing the program counter, fetched instruction, synchronisation signals and colour outputs.
- To program the Nexys A7-100T and confirm correct output on a physical VGA monitor.
- To analyse on-chip power with Vivado and to compare the power of the earlier build with the current design.

The remainder of this report is organised as follows. Chapter 2 surveys related work, Chapter 3 states the problem, Chapter 4 compares existing and proposed systems, Chapter 5 describes the methodology and the hardware modules, Chapter 6 lists the hardware and software requirements, Chapter 7 presents the simulation, hardware and power results, and the report ends with the conclusion, future scope and references.

CHAPTER 2

LITERATURE SURVEY

2.1 The RISC-V Instruction Set Manual, Volume I: User-Level ISA

AUTHORS: Andrew Waterman, Yunsup Lee, David A. Patterson and Krste Asanović

PUBLISHED ON: 2014

This technical report specifies the RISC-V base integer ISAs (RV32I and RV64I) and the standard extensions for multiplication and division, atomics, floating point and compressed instructions. The base ISA uses fixed 32-bit instructions, a load/store model and 32 integer registers, of which register x0 is hard-wired to zero.

Advantages:

- Simple and regular instruction encoding that eases decoder design
- Modular extensions allow a design to include only what it needs

Disadvantages:

- Base ISA has no multiply, divide or floating-point support
- No graphics-specific instructions

Limitation:

Defines the architecture only; it does not define the micro-architecture or memory-mapped peripherals.

Future Scope:

Memory-mapped accelerators and custom extensions for domain-specific workloads such as graphics.

2.2 Instruction Sets Should Be Free: The Case for RISC-V

AUTHORS: Krste Asanović and David A. Patterson

PUBLISHED ON: 2014

This report argues for an open, royalty-free ISA, comparing the technical and commercial consequences of proprietary instruction sets with those of an open standard, and motivates the creation of RISC-V.

Advantages:

- No licensing cost for academic and commercial implementations
- Encourages a shared ecosystem of open cores, tools and software

Disadvantages:

- Ecosystem and tool maturity were limited when the report was written

Limitation:

It is a position paper and does not present implementation or measurement results.

Future Scope:

Open-source hardware implementations with complete verification and peripheral support.

2.3 The Rocket Chip Generator

AUTHORS: Krste Asanović et al.

PUBLISHED ON: 2016

The Rocket Chip generator is a parameterised SoC generator written in Chisel. It produces in-order Rocket cores with caches, a TileLink interconnect and peripheral interfaces, and has been taped out in silicon.

Advantages:

- Highly parameterisable and silicon proven
- Supports attachment of custom accelerators

Disadvantages:

- Large and complex code base
- Chisel tool flow adds a learning curve

Limitation:

Too heavy for small teaching or single-purpose FPGA designs.

Future Scope:

Lightweight cores with custom accelerators attached through a simple memory-mapped interface.

2.4 Slow and Steady Wins the Race? A Comparison of Ultra-Low-Power RISC-V Cores for IoT Applications

AUTHORS: Pasquale Davide Schiavone et al.

PUBLISHED ON: 2017

This work compares two open ultra-low-power cores from the PULP project, RI5CY and zero-riscy, in terms of area, performance and power, showing how micro-architectural choices trade performance against energy.

Advantages:

- Quantifies area, speed and power trade-offs of RISC-V cores
- Provides open-source reference cores

Disadvantages:

- Evaluation is centred on IoT workloads, not graphics

Limitation:

Results depend on the technology and benchmarks chosen.

Future Scope:

Evaluating small cores together with workload-specific accelerators.

2.5 Near-Threshold RISC-V Core with DSP Extensions for Scalable IoT Endpoint Devices

AUTHORS: Michael Gautschi et al.

PUBLISHED ON: 2017

This paper presents the RI5CY core, which extends RISC-V with hardware loops, post-incrementing load/store and SIMD instructions, and demonstrates operation at near-threshold voltage for high energy efficiency.

Advantages:

- Improved energy efficiency for signal-processing kernels
- Shows the value of ISA extensions for data-parallel work

Disadvantages:

- Extensions require compiler and tool-chain support

Limitation:

Parallelism is obtained inside the core rather than through a separate accelerator.

Future Scope:

Combining lightweight extensions with memory-mapped accelerators.

2.6 PicoRV32: A Size-Optimised RISC-V CPU

AUTHORS: Claire Wolf (open-source project)

PUBLISHED ON: 2015

PicoRV32 is a compact, configurable RV32I[M][C] core with a simple native memory interface, intended to be used as an auxiliary processor inside FPGA designs.

Advantages:

- Very small and easy to integrate
- Configurable features and a simple bus interface

Disadvantages:

- Multi-cycle execution gives low instructions-per-cycle

Limitation:

Not designed for high throughput computation on its own.

Future Scope:

Pairing small cores with hardware accelerators for compute- or graphics-intensive tasks.

2.7 Vortex: Extending the RISC-V ISA for GPGPU and 3D Graphics

AUTHORS: Fares Elsabbagh et al.

PUBLISHED ON: 2021

Vortex is an open-source GPGPU built on RISC-V. It adds a small set of instructions for SIMT execution and 3D graphics and was demonstrated on FPGA running OpenCL workloads.

Advantages:

- Open-source and scalable GPU architecture
- Shows RISC-V can be extended for graphics and GPGPU

Disadvantages:

- High FPGA resource use
- Complex software stack

Limitation:

Targets larger devices and workloads than a basic display system on a mid-range FPGA.

Future Scope:

Reduced-footprint variants with a direct frame-buffer display path.

2.8 FlexGrip: A Soft GPGPU for FPGAs

AUTHORS: Kevin Andryc, Murtaza Merchant and Russell Tessier

PUBLISHED ON: 2013

FlexGrip is a soft GPGPU architecture for FPGAs that executes compiled CUDA kernels without re-synthesis, using a streaming-multiprocessor style design with multiple scalar processors.

Advantages:

- Runs compiled GPU kernels on an FPGA
- Scalable number of processing elements

Disadvantages:

- Consumes substantial logic and block RAM

Limitation:

Focus is general-purpose computing, not display output.

Future Scope:

Integration of graphics output and lower-power operation.

2.9 Enabling GPGPU Low-Level Hardware Explorations with MIAOW

AUTHORS: Raghuraman Balasubramanian et al.

PUBLISHED ON: 2015

MIAOW is an open-source RTL implementation of the compute units of a commercial GPU architecture. It enables hardware-level exploration of GPU behaviour, area and power.

Advantages:

- Open RTL enables detailed architecture studies
- Validated against real GPU benchmarks

Disadvantages:

- Very large design, not suited to small FPGAs

Limitation:

No graphics pipeline or display controller is included.

Future Scope:

Simplified accelerators that keep the key parallelism at far lower cost.

2.10 Nyami: A Synthesizable GPU Architectural Model for General-Purpose and Graphics-Specific Workloads

AUTHORS: Jeff Bush, Philip Dexter, Timothy N. Miller and Aaron Carpenter

PUBLISHED ON: 2015

Nyami presents a synthesisable GPU architecture model with multithreaded vector processing that can run both general-purpose and graphics workloads, used to study the effect of architectural parameters.

Advantages:

- Synthesisable and parameterisable
- Supports graphics-specific workloads

Disadvantages:

- Complex design and tool-chain

Limitation:

Resource usage is high for simple frame-buffer applications.

Future Scope:
Compact graphics units suited to teaching and embedded SoCs.

2.11 GPU Computing

AUTHORS: John D. Owens et al.

PUBLISHED ON: 2008

This survey in the Proceedings of the IEEE explains the architecture of programmable GPUs and why data-parallel workloads map efficiently onto many simple processing elements.

Advantages:

- Clear explanation of data-parallel hardware
- Basis for modern accelerator design

Disadvantages:

- Describes large discrete GPUs with high power

Limitation:
Does not address small embedded or FPGA-based accelerators.

Future Scope:
Scaling down data-parallel ideas for low-power embedded graphics.

2.12 FPGA Prototyping by Verilog Examples

AUTHORS: Pong P. Chu

PUBLISHED ON: 2008

This book presents a systematic set of Verilog designs for FPGA prototyping, including video interface and VGA controller designs with frame buffers, pixel generation and synchronisation signals.

Advantages:

- Practical, synthesisable examples
- Detailed treatment of VGA timing and video interfaces

Disadvantages:

- Targets older FPGA families and boards

Limitation:
Does not combine the video path with a RISC-V processor.

Future Scope:
Combining the VGA design methodology with open ISA processors on modern FPGAs.

2.13 VESA Monitor Timing Standard (DMT)

AUTHORS: Video Electronics Standards Association (VESA)

PUBLISHED ON: 2013

The Display Monitor Timing standard defines the timing of video modes. The 640 × 480 at 60 Hz mode uses a 25.175 MHz pixel clock, 800 × 525 total pixels, and negative-polarity horizontal and vertical synchronisation pulses.

Advantages:

- Universal compatibility with VGA monitors
- Exact porch and sync timing values

Disadvantages:

- Fixed list of modes

Limitation:

Defines timing only and not how the frame content is generated.

Future Scope:

Higher resolutions using clock-management primitives for exact pixel clocks.

2.14 Measuring the Gap Between FPGAs and ASICs

AUTHORS: Ian Kuon and Jonathan Rose

PUBLISHED ON: 2007

This paper measures the area, delay and power overhead of FPGA implementations compared with standard-cell ASICs and shows how embedded memory and multiplier blocks reduce the gap.

Advantages:

- Quantitative basis for FPGA versus ASIC decisions
- Highlights the benefit of hard blocks such as BRAM and DSP

Disadvantages:

- Based on older technology nodes

Limitation:

Does not analyse specific application designs.

Future Scope:

Using hard blocks and careful architecture to reduce FPGA power.

2.15 Power Estimation Techniques for FPGAs

AUTHORS: Jason H. Anderson and Farid N. Najm

PUBLISHED ON: 2004

This paper presents techniques for estimating FPGA power at different design stages, based on switching activity, capacitance and resource usage, in the same spirit as vendor early-power estimators.

Advantages:

- Enables early power estimation before implementation
- Models logic, routing, clock and I/O contributions

Disadvantages:

- Accuracy depends on the quality of the activity data

Limitation:

Early estimates can differ from post-implementation results.

Future Scope:

Activity-driven estimation using simulation data.

2.16 Dynamic Power Consumption in Virtex-II FPGA Family

AUTHORS: Li Shang, Alireza S. Kaviani and Kusuma Bathala

PUBLISHED ON: 2002

This work analyses where dynamic power is consumed in an FPGA family and shows that interconnect, logic, clocking and I/O each contribute significantly, guiding power-aware design.

Advantages:

- Identifies the dominant sources of dynamic power
- Guides design-level power optimisation

Disadvantages:

- Measured on an older device family

Limitation:

Results depend on device and design style.

Future Scope:

Power-aware design and tool flows for current FPGA families.

CHAPTER 3

PROBLEM STATEMENT

Graphics output is a data-parallel task, yet many small embedded and FPGA-based processor systems still produce images entirely in software. A scalar processor that writes every pixel of a frame individually spends many instruction cycles per pixel, so even a simple operation such as filling a rectangle occupies the processor for a long time and leaves little capacity for other work.

A second problem is the **absence of a verified display path**. A VGA display requires exact horizontal and vertical synchronisation signals, correct blanking and a pixel clock close to the nominal value. Small mistakes in these timings result in a blank, unstable or mis-positioned image. Designs that are only simulated or that have no testbench frequently fail when first connected to a monitor.

Third, a frame buffer for even a modest resolution needs a large amount of memory. A 320 × 240 image with 12 bits per pixel needs 921,600 bits. If this memory is not described in a form that the synthesis tool maps onto block RAM, it consumes logic resources and power unnecessarily.

Fourth, **power consumption** is rarely considered early in student and prototype designs. Dynamic power from I/O, logic and signal switching can be several watts on an FPGA, and without analysis it is not known which part of the design is responsible or whether design changes actually reduce power.

Finally, a complete system requires the processor, the register interface, the graphics accelerator, the frame buffer and the video controller to be integrated with consistent reset and clocking, and to be verified both in simulation and on hardware.

Problem statement: To design and implement a 32-bit RISC-V processor based system with a parallel graphics accelerator, a memory-mapped GPU register interface, an on-chip frame buffer and a VGA controller on the Nexys A7-100T FPGA; to verify it through simulation and on a physical monitor; and to analyse and reduce its on-chip power consumption.

CHAPTER 4

EXISTING AND PROPOSED SYSTEM

4.1 EXISTING SYSTEM

In most small processor-based FPGA systems the processor itself generates the image. Software running on the core computes each pixel and stores it into a frame buffer or directly to a video peripheral, and a fixed VGA controller scans the memory to the monitor. This approach is simple but ties graphics performance to processor speed.

Larger systems use vendor video IP such as AXI-based video timing controllers and frame-buffer DMA blocks with soft processors, or open GPU designs such as Vortex, FlexGrip and MIAOW. These provide high performance but require substantial logic, block RAM, tool-chain support and power, and are often larger than the application needs.

Many teaching designs also provide a stand-alone VGA test pattern generator without any processor control, which verifies the monitor interface but does not demonstrate a processor-controlled graphics path.

4.1.1 DISADVANTAGES OF EXISTING SYSTEM

i. **Software-limited graphics:** The processor writes every pixel, which wastes cycles and energy on repetitive work.

ii. **Large resource footprint:** Full GPU or vendor video IP consumes large logic and memory resources that a simple display system does not need.

iii. **No integrated processor–accelerator interface:** Test pattern generators have no register interface through which software can control graphics operations.

iv. **Unverified timing:** Display paths without a testbench and hardware check often fail on a real monitor.

v. **Unmanaged power:** Power is seldom estimated or compared across design revisions, so excess I/O and logic power goes unnoticed.

vi. **Complex tool-chains:** Large GPU designs require complicated software stacks that are impractical for an academic prototype.

4.2 PROPOSED SYSTEM

The proposed system is a compact, modular SoC in which a 32-bit RISC-V processor commands a parallel graphics accelerator through memory-mapped registers, and a block-RAM frame buffer feeds a VGA controller. It is organised in the following layers:

1. Processor Layer

A 32-bit RISC-V core fetches instructions from an instruction ROM and executes the control program. Stores to a reserved address window are routed to the GPU register block.

2. GPU Register Interface (MMIO)

Four memory-mapped registers (command, argument 0, argument 1 and colour) starting at address 0x4000_0000 let software describe a drawing operation and trigger it with a start bit.

3. Parallel Graphics Accelerator

A parameterised pixel datapath (12-bit pixel width by default) takes pixel coordinates, a colour and an enable signal and produces a valid pixel. Its combinational structure is the unit that is replicated to generate several pixels in parallel.

4. Frame Buffer

A 320 × 240 frame buffer with 12 bits per pixel is described so that Vivado maps it onto block RAM. It has a write port for pixel updates and a registered read port for display scanning.

5. VGA Controller and Pixel Clock Enable

A divide-by-four enable derives a 25 MHz pixel rate from the 100 MHz clock. Horizontal and vertical counters generate active-low synchronisation pulses and blank the colour outputs outside the visible 640 × 480 area.

6. Verification and Power Analysis Layer

A testbench drives clock and reset, waveforms are inspected in Vivado simulation, the design is programmed to the board for visual verification, and Vivado power reports are generated and compared.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

i. **Hardware pixel generation:** Graphics work is offloaded from the processor to a dedicated datapath.

ii. **Open and modular architecture:** The RISC-V base ISA and separate modules allow each block to be modified or replaced independently.

iii. **Low resource footprint:** The frame buffer maps to block RAM and the design uses a small amount of logic.

iv. **Standard-compliant display:** 640 × 480 timing with active-low synchronisation at approximately 59.5 Hz is accepted by an ordinary VGA monitor.

v. **Simple software interface:** Four registers are enough to describe and start a drawing operation.

vi. **Verified on hardware:** The design is validated in simulation and on a physical monitor.

vii. **Power awareness:** On-chip power is reported and compared between design revisions.

viii. **Scalable:** Additional accelerator lanes or larger frame buffers can be added within the available device resources.

CHAPTER 5

METHODOLOGY

The methodology follows a hardware design flow: define the specification, partition the system into synthesisable Verilog modules, verify the modules together in simulation, constrain and implement the design on the Nexys A7-100T, observe the output on a monitor and finally analyse on-chip power. All modules are written in synthesisable Verilog and are targeted at the Xilinx Artix-7 XC7A100T-1CSG324 device.

5.1 System Overview

Figure 5.1 shows the system. The command path consists of the instruction ROM, the RISC-V core, the GPU register block and the parallel graphics accelerator. The display path consists of the frame buffer, the read-address generator, the pixel clock enable and the VGA controller. In the validated build, a demonstration writer (module riscv_gpu_soc) fills the frame buffer with a red rectangle on reset, which allows the complete display path to be verified independently of the processor program. The planned integration connects the accelerator's pixel output to the frame-buffer write port.

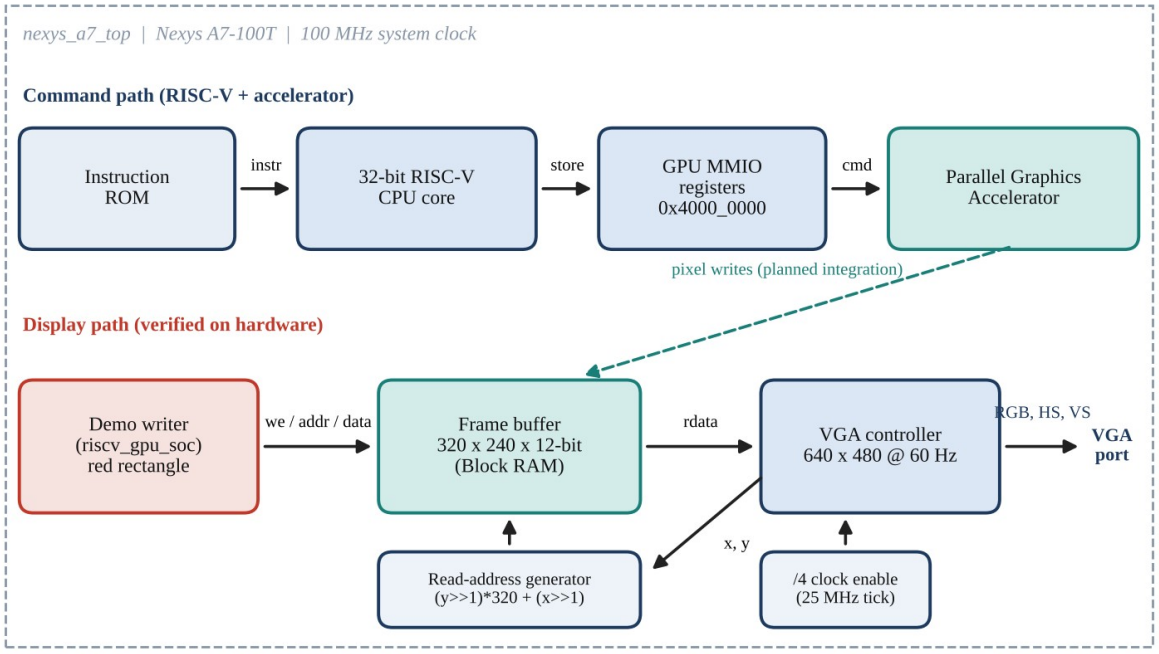


Fig 5.1 System Architecture and Methodology Block Diagram

5.2 Design Specification

The key design parameters are summarised in Table 5.1.

Table 5.1 Design Parameters

Parameter	Value
Target board / device	Digilent Nexys A7-100T, Xilinx Artix-7 XC7A100T-1CSG324

Parameter	Value
System clock	100 MHz (pin E3, period 10 ns)
Pixel enable	Every 4th clock, approx. 25 MHz (25.175 MHz nominal)
Display mode	640 × 480 visible, 800 × 525 total, approx. 59.5 Hz
Synchronisation	HSYNC and VSYNC active low
Frame buffer	320 × 240 pixels, RGB444 (12 bits), 76,800 words
Display scaling	2 × pixel replication in both directions
Colour output	4 bits each for red, green and blue (4,096 colours)
Reset	CPU_RESETN push-button, active low (pin C12)

5.3 RISC-V Processor and Instruction ROM

The processor is a 32-bit RISC-V core that executes the base integer instructions. Instructions are supplied by a read-only instruction memory (module instruction_rom) addressed by the program counter. The ROM is word addressed using bits [7:2] of the address and contains 64 words; the first locations hold NOP instructions (0x00000013, addi x0, x0, 0), and all other locations return 0x0000006F, which is the encoding of jal x0, 0, a jump to the same address. A program that has finished therefore remains in a safe idle loop. The program counter (pc), the fetched instruction (instr) and a status word are brought out for observation in simulation, as shown in Section 7.1.

5.4 GPU Register Interface (MMIO)

The module gpu_mmio decodes stores whose address lies in the window 0x4000_0000 to 0x4000_000F (address bits [31:4] equal to 0x4000000). Four 32-bit registers are provided (Table 5.2). Writing the command register latches the command and generates a single-cycle start strobe taken from bit 0 of the written data. A status register mirrors the start strobe so that software can observe that a command was accepted. All registers are cleared by reset.

Table 5.2 GPU Register Map

Address	Register	Function
0x4000_0000	COMMAND	Command word; bit 0 = start strobe (one clock cycle)
0x4000_0004	ARG0	First argument (for example position)
0x4000_0008	ARG1	Second argument (for example size)
0x4000_000C	COLOR	12-bit RGB444 colour value
(internal)	STATUS	Mirrors the start strobe; read by software

5.5 Parallel Graphics Accelerator

The module parallel_graphics_accelerator is the pixel datapath. Its parameter PIXEL_WIDTH sets the colour width (12 by default). For each pixel it receives the screen coordinates x and y

(10 bits each), a colour and an enable signal, and it outputs pixel_valid (equal to enable) and pixel_data (the colour when enabled and zero otherwise). Because the datapath is purely combinational and contains no shared state, several instances can operate side by side, each handling a different pixel, which is the basis of the parallel graphics accelerator. The present build instantiates this single lane as the building block; the lanes are driven by the command registers once the processor path is connected to the frame buffer.

5.6 Frame Buffer

The frame buffer (module framebuffer) stores 320 × 240 = 76,800 pixels of 12 bits each, i.e. 921,600 bits (112.5 KiB). The 17-bit address bus covers the 76,800 locations. The memory has one synchronous write port and one synchronous read port with a registered output, a structure that Vivado infers as block RAM; the contents are initialised to zero. The block RAM requirement, about 0.92 Mb of the 4.86 Mb available on the XC7A100T, is roughly 19 % of the device's block memory.

5.7 VGA Timing Controller

The VGA controller keeps two counters, vga_x and vga_y, that advance on the pixel enable. The horizontal counter runs from 0 to 799 and the vertical counter from 0 to 524. The timing parameters used are given in Table 5.3 and drawn in Figure 5.2. Because the pixel enable is one quarter of the 100 MHz clock the pixel rate is exactly 25 MHz, 0.7 % below the nominal 25.175 MHz; the line rate is therefore 31.25 kHz and the frame rate 59.52 Hz, which standard monitors accept.

Table 5.3 VGA Timing Parameters (640 × 480)

Direction	Visible	Front porch	Sync pulse	Back porch
Horizontal (pixels)	640	16	96	48
Vertical (lines)	480	10	2	33
Total	800 / 525			
Polarity	n/a	n/a	Negative	n/a

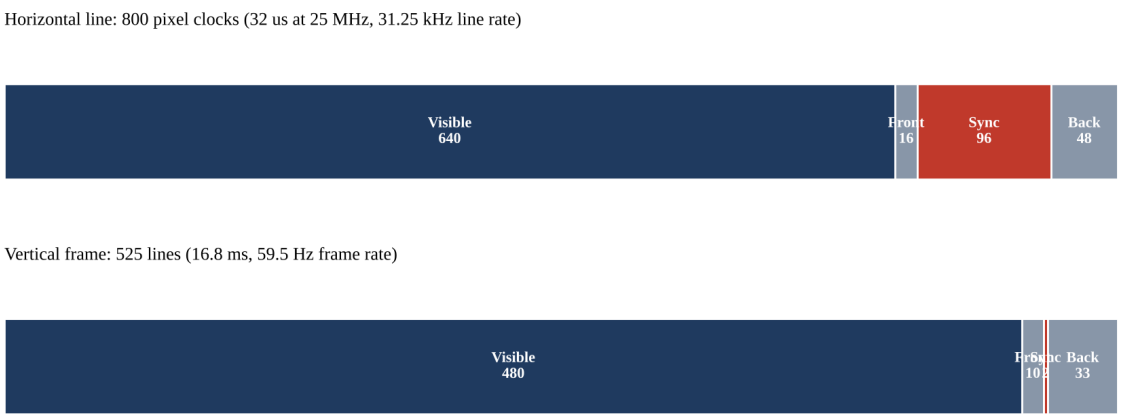


Fig 5.2 VGA Horizontal and Vertical Timing

The visible flag is asserted while vga_x < 640 and vga_y < 480. The red, green and blue outputs equal the four bits of the frame-buffer pixel inside the visible area and are forced to zero outside it, which is the blanking required by the monitor. The synchronisation outputs are asserted low during the sync interval of each direction.

5.8 Read-Address Generation and Display Scaling

The frame buffer is 320 × 240 while the display is 640 × 480, so each frame-buffer pixel is shown as a 2 × 2 block. The read address is computed as (y >> 1) × 320 + (x >> 1), where the right shift implements the replication. The multiplication by the constant 320 can be implemented in logic or in a DSP48E1 slice; the DSP entry in the power report of the current design is consistent with this use. As the frame buffer has a registered read output, the pixel arrives one clock after its address, which is a constant shift of the picture that is not visible at 25 MHz.

5.9 Demonstration Writer

To validate the display path independently of the processor program, the module riscv_gpu_soc acts as a writer. After reset it scans the frame buffer from address 0 to 76,799 and writes one pixel per clock: the colour 0xF00 (red) inside the rectangle x = 80 to 239, y = 60 to 179, and 0x000 (black) elsewhere. The whole frame is written in 76,800 cycles (768 µs at 100 MHz) after which the writer stops. With 2 × scaling the rectangle appears at x = 160 to 479, y = 120 to 359 on the monitor, a rectangle of 320 × 240 pixels centred on the screen (Figure 5.3).

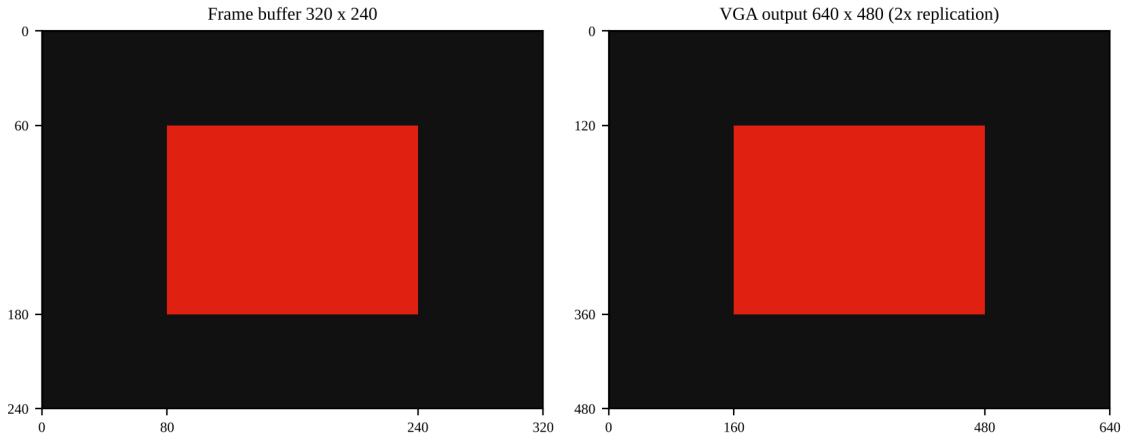


Fig 5.3 Frame Buffer Contents and Resulting VGA Output

5.10 Simulation Setup

The testbench (tb_nexys_a7_top) generates the 100 MHz clock with a 10 ns period, holds CPU_RESETN low for 200 ns, releases it and runs for 18 ms, which is longer than one frame (16.8 ms). The signals CLK100MHZ, CPU_RESETN, VGA_HS, VGA_VS and VGA_R/G/B are added to the waveform window of the Vivado behavioural simulator, and internal signals such as the program counter are observed through the hierarchy. The simulation is advanced with the Run for 1 ms control.

5.11 Constraints and Implementation

The top module nexys_a7_top is implemented with a constraint file that defines the 100 MHz clock (create_clock, period 10.000 ns) and assigns the clock, reset and VGA pins with the LVCMOS33 I/O standard (Table 5.4). The flow is synthesis, implementation, bitstream generation and programming through the Vivado Hardware Manager.

Table 5.4 Pin Assignments Used in the Constraint File

Signal	Pin	Signal	Pin
CLK100MHZ	E3	VGA_B[0]	D7
CPU_RESETN	C12	VGA_B[1]	C7
VGA_R[0]	A3	VGA_B[2]	A8
VGA_R[1]	C5	VGA_B[3]	D8
VGA_R[2]	A4	VGA_G[0]	B4
VGA_R[3]	C6	VGA_G[1]	A6
VGA_HS	B11	VGA_G[2] / [3]	B6 / A7
VGA_VS	B12		

5.12 Hardware Validation

After bitstream generation the Nexys A7-100T is connected to the host computer by USB and to a VGA monitor by a VGA cable. The device is programmed from the Hardware Manager, the

reset button is released, and the monitor is observed. A correct implementation shows a centred red rectangle on a dark background.

5.13 Power Analysis Method

On-chip power is evaluated with the Vivado power analysis tool, which reports static power, dynamic power and the split of dynamic power into signals, logic, block RAM, DSP and I/O. The dynamic power of CMOS logic follows $P = \alpha \cdot C \cdot V^2 \cdot f$, where α is the switching activity, C the switched capacitance, V the supply voltage and f the frequency, so power falls when fewer nets switch, when less logic and fewer I/O pins are driven, and when slower clocks are used. Power is reported from the synthesised netlist (early estimate) and from the implemented design (post place-and-route). With vectorless activity estimation the tool gives a low confidence level, so the figures are used to compare design revisions on an equal basis rather than as absolute measurements. The results are presented in Section 7.3.

5.14 System Outcome

The methodology yields a complete, simulated and hardware-validated display system consisting of the frame buffer, VGA controller, clock enable, GPU register interface, pixel datapath and instruction ROM, together with a power analysis that quantifies the reduction obtained in the current design.

CHAPTER 6
HARDWARE AND SOFTWARE REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

i. **FPGA Board:** Digilent Nexys A7-100T with the Xilinx Artix-7 XC7A100T-1CSG324 device, 100 MHz oscillator, VGA port and CPU reset push-button.

ii. **VGA Monitor and Cable:** A monitor that accepts 640 × 480 at 60 Hz (an HP LV1911 monitor is used) with a standard VGA cable.

iii. **Host Computer:** A laptop or desktop with at least 16 GB RAM is recommended for running Vivado synthesis, implementation and simulation.

iv. **USB Cable:** USB (micro-B) cable for power, programming and JTAG communication between the host and the board.

v. **Storage:** At least 100 GB of free disk space for the Vivado installation and project files.

6.2 SOFTWARE REQUIREMENTS

i. **Operating System:** Windows 10/11 or a supported Linux distribution.

ii. **Design Suite:** AMD (Xilinx) Vivado Design Suite (version 2025.2 used) for synthesis, implementation, simulation, power analysis and Hardware Manager.

iii. **Hardware Description Language:** Verilog HDL for all RTL and for the testbench.

iv. **Simulator:** Vivado Simulator (XSim) for behavioural simulation and waveform viewing.

v. **Constraints:** Xilinx Design Constraints (XDC) file for pins, I/O standards and clock.

vi. **Documentation Tools:** Reference manual of the Nexys A7 board and the AMD user guides for 7-series devices.

CHAPTER 7

RESULTS

7.1 Behavioural Simulation Results

The design was simulated in Vivado as described in Section 5.10. Figure 7.1 shows the processor-related signals: the program counter pc holds 0x00000034, the fetched instruction instr is 0x0000006F (jal x0, 0, the idle self-jump) and the status register is 0x00000000, while VGA_R, VGA_G and VGA_B read 0 at this point of the frame. The short green region at the start of the pc and instr traces is the interval in which these values change rapidly after reset; afterwards both remain constant.

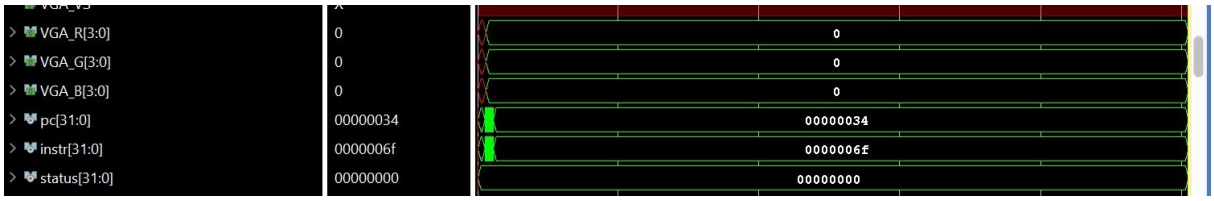


Fig 7.1 Processor Signals: pc, instr and status

Figure 7.2 shows the display signals around t = 9.0 ms. VGA_HS produces a periodic pulse for every scan line and VGA_VS stays high. VGA_R toggles between 0xF and 0x0 within the lines as the scan crosses the rectangle (all four bits [3:0] move together), while VGA_G stays at 0 (and VGA_B at 0). This confirms that the frame buffer holds a red-only image and that the colour outputs change only inside the visible window.

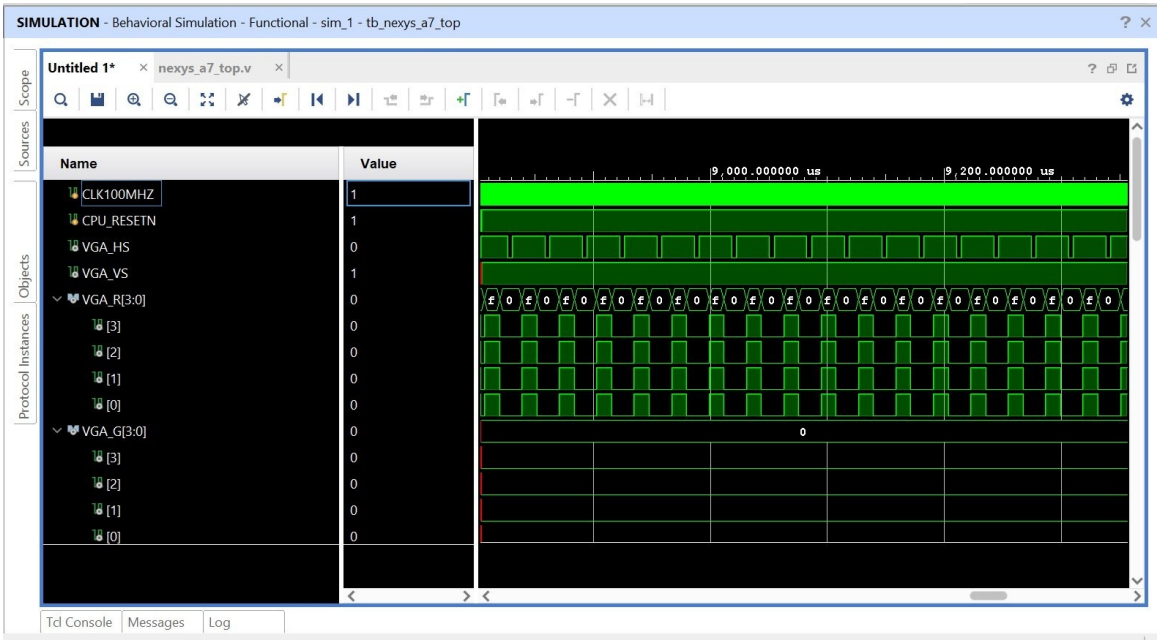


Fig 7.2 Simulation Waveform of VGA Synchronisation and Colour Outputs

Figures 7.3 and 7.4 show the simulation window at 1 ms and 4 ms, advanced with the Run for 1 ms control. At 1 ms the scan is at about line 31 of the frame, above the rectangle (which starts at line 120), and at 4 ms the scan is at the very start of a line, to the left of the rectangle; in both cases VGA_R, VGA_G and VGA_B are 0 and VGA_HS and VGA_VS are 1, which is the expected value outside the rectangle.

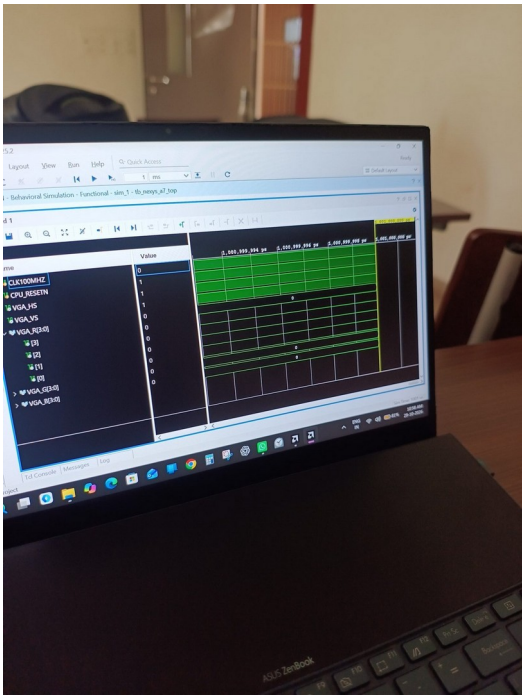


Fig 7.3 Simulation at 1 ms

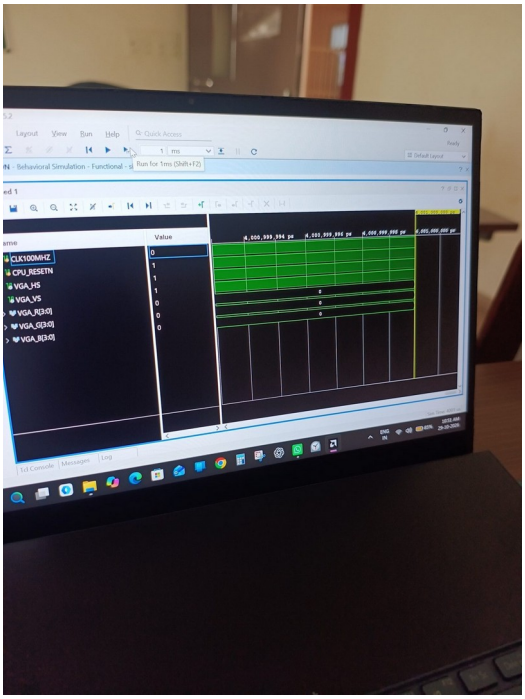


Fig 7.4 Simulation at 4 ms

Table 7.1 Verification Summary

Check	Expected	Observed
Reset release	Design starts after CPU_RESETN = 1 at 200 ns	Outputs defined after reset (Fig 7.2)
HSYNC	Active-low pulse once per line (32 µs)	Regular pulse train (Fig 7.2)
VSYNC	High outside the 2-line sync interval	High at 1 ms, 4 ms and 9 ms
Colour outside rectangle	R = G = B = 0	All 0 at 1 ms and 4 ms
Colour inside rectangle	R = 0xF, G = B = 0	R toggles 0 ↔ F; G, B = 0
Processor idle	pc stable, instr = 0x0000006F	pc = 0x34, instr = 0x6F

7.2 Hardware Results

The bitstream was loaded onto the Nexys A7-100T through the Vivado Hardware Manager, with the board connected to the laptop by USB and to the HP LV1911 monitor by a VGA cable. Figures 7.5 and 7.6 show the complete set-up and Figure 7.7 a close view of the monitor. A solid red rectangle is displayed in the middle of the screen, occupying about half of the width and half of the height, exactly the 320 × 240 pixel region predicted in Section 5.9. The area around the rectangle is dark (it appears blue-tinted in the photographs). The edges of the

rectangle are sharp, which shows that the horizontal and vertical counters and the 2 × scaling work as designed, and the stable picture shows that the monitor locks to the generated 640 × 480 at 59.5 Hz synchronisation signals.

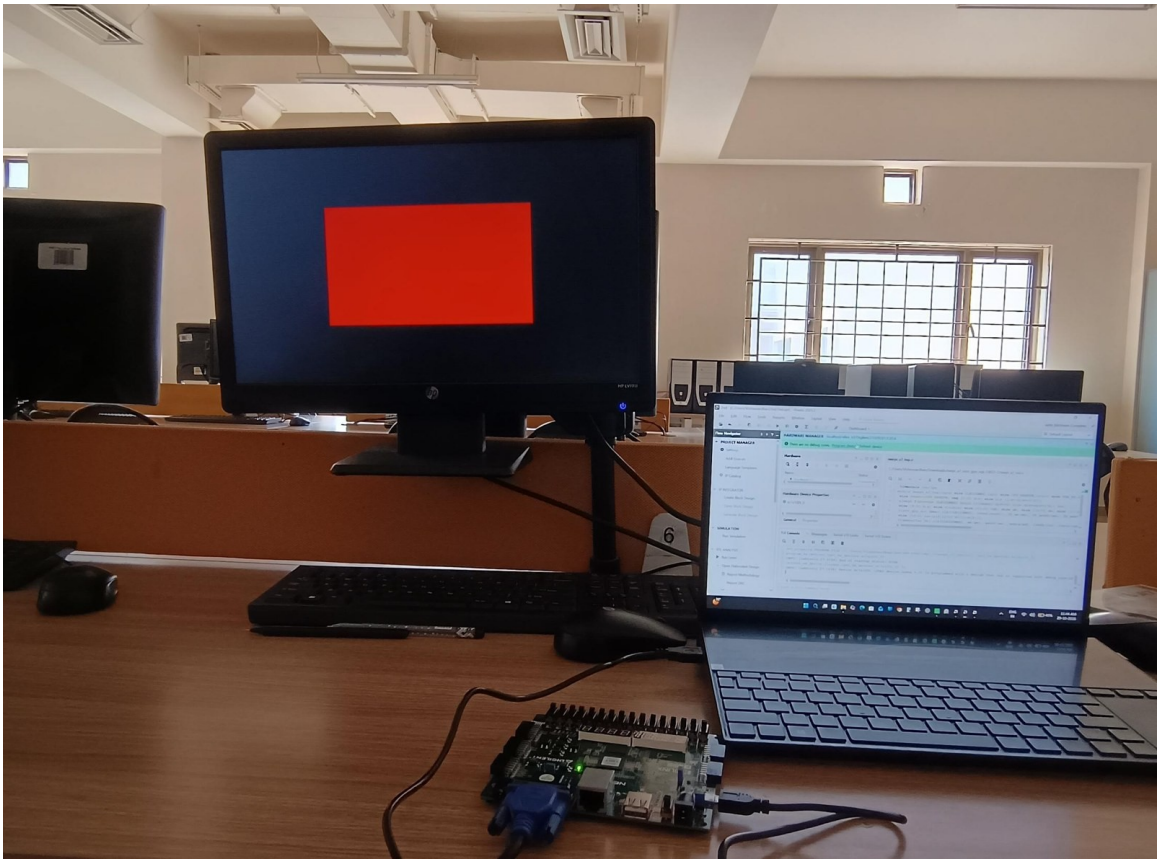


Fig 7.5 Hardware Set-up: Nexys A7-100T Driving the VGA Monitor

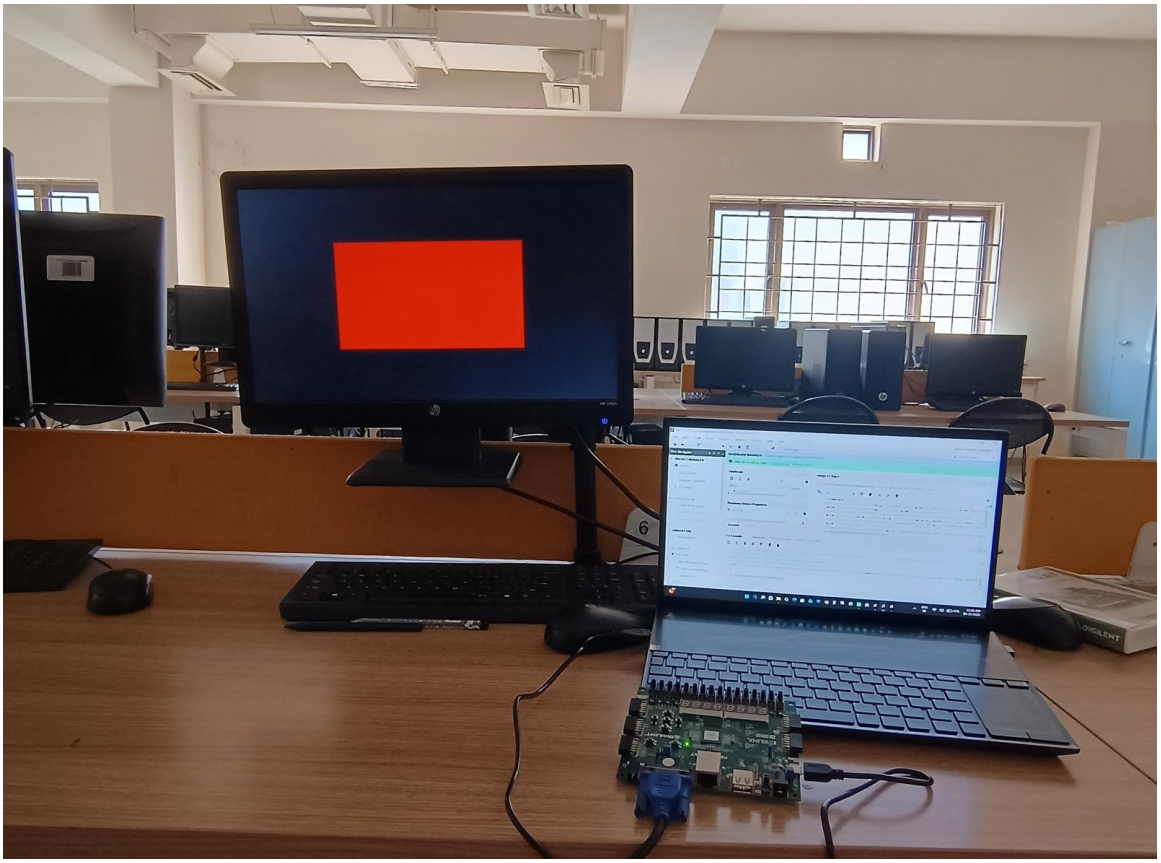


Fig 7.6 Hardware Output (Second View of the Set-up)

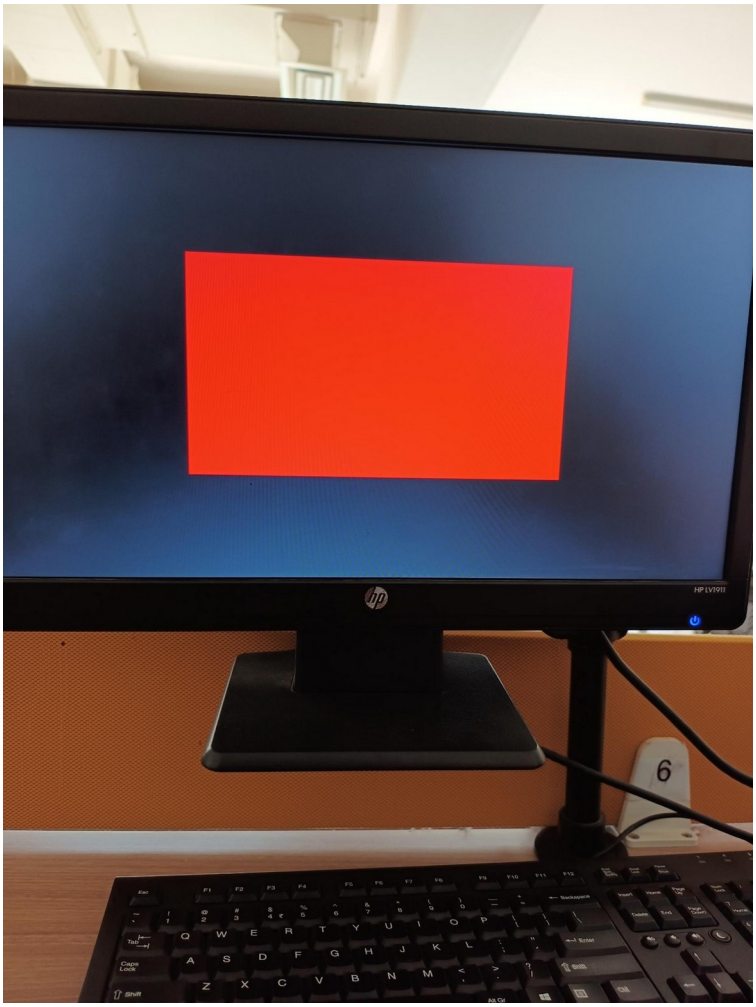


Fig 7.7 Close-up of the Red Rectangle on the Monitor

7.3 Power Analysis

Vivado power reports were generated for two builds of the design. Figure 7.8 shows the power summary of the current design computed from the synthesised netlist (total on-chip power 2.675 W), and Figure 7.9 shows the summary of the implemented design of the earlier build (total on-chip power 6.098 W). Both use the typical process, 25 °C ambient temperature, an effective θJA of 4.6 °C/W and report a low confidence level because the switching activity is derived from constraints and vectorless analysis.

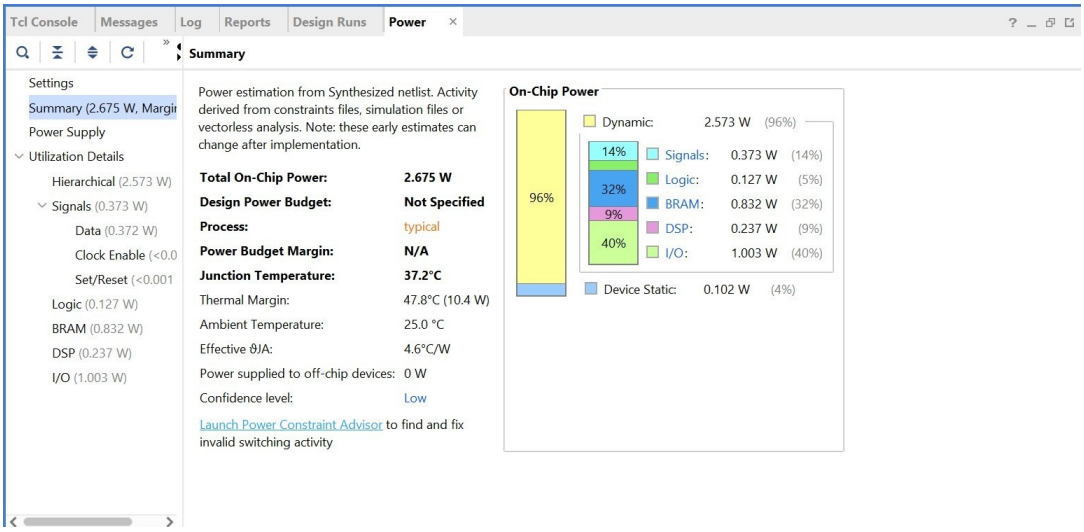


Fig 7.8 Power Report: Current Design (Synthesised Netlist Estimate)

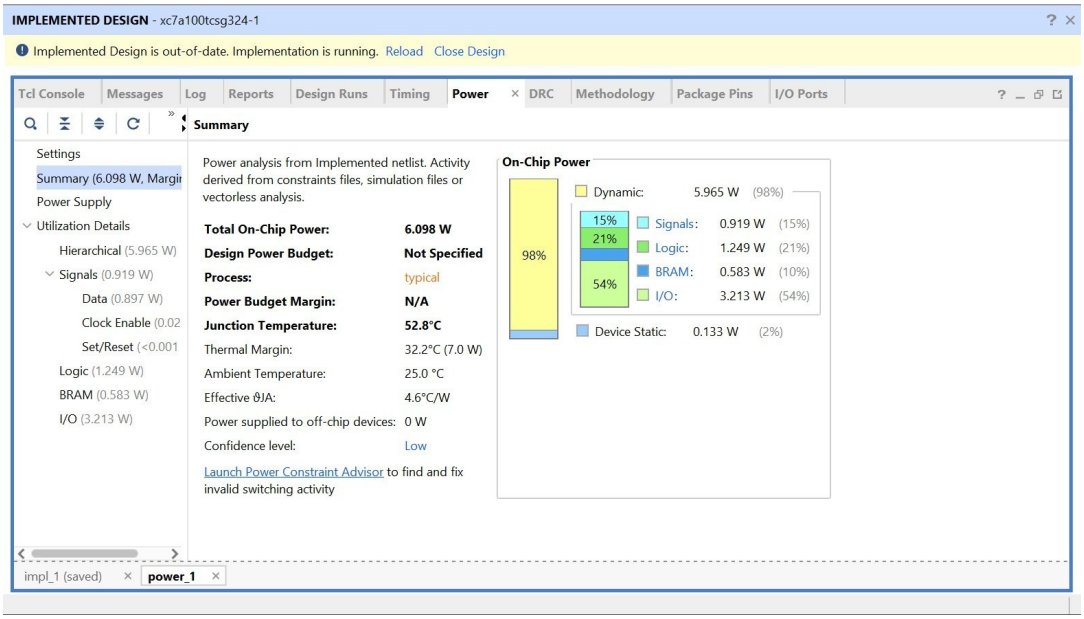


Fig 7.9 Power Report: Earlier Build (Implemented Design)

7.4 Power Comparison and Power Reduction

Table 7.2 and Figure 7.10 compare the two reports. The total on-chip power falls from **6.098 W to 2.675 W**, a reduction of **3.423 W (56.1 %)**. The dynamic power falls by 3.392 W (56.9 %) and device static power by 0.031 W (23.3 %), and the junction temperature drops from 52.8 °C to 37.2 °C.

Table 7.2 Power Comparison

Parameter	Earlier build (implemented)	Current design (synthesised)	Change
Total on-chip power	6.098 W	2.675 W	−3.423 W (−56.1 %)
Dynamic power	5.965 W	2.573 W	−3.392 W (−56.9 %)
Device static power	0.133 W	0.102 W	−0.031 W (−23.3 %)
Signals	0.919 W	0.373 W	−0.546 W (−59.4 %)
Logic	1.249 W	0.127 W	−1.122 W (−89.8 %)
Block RAM	0.583 W	0.832 W	+0.249 W (+42.7 %)
DSP	0.000 W (not used)	0.237 W	+0.237 W
I/O	3.213 W	1.003 W	−2.210 W (−68.8 %)
Junction temperature	52.8 °C	37.2 °C	−15.6 °C
Thermal margin	32.2 °C (7.0 W)	47.8 °C (10.4 W)	+15.6 °C (+3.4 W)

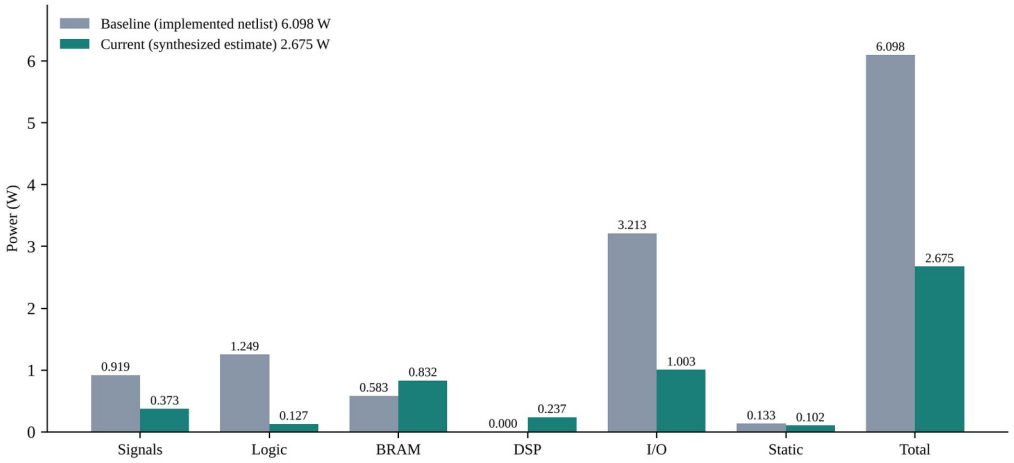


Fig 7.10 Power by Category: Earlier Build versus Current Design

The reduction comes mainly from I/O (−2.210 W, 65 % of the total saving) and logic (−1.122 W, 33 %), followed by signal power (−0.546 W). I/O is the largest remaining contributor at 40 % of the current total, followed by block RAM at 32 %. Block RAM and DSP power increase by 0.249 W and 0.237 W because the frame buffer and the address multiplication now use these dedicated blocks, which are far more efficient than the equivalent LUT logic they replace (logic power falls by almost 90 %). Using the hard blocks in this way is the same trade-off reported in the FPGA literature [14].

Note: the earlier build figure is a post-implementation report whose window showed that implementation was being re-run, and the current figure is a post-synthesis estimate; both have low confidence. The comparison therefore shows the trend and relative contributions. A final comparison should be repeated after the current design has been fully implemented.

CONCLUSION

This report presented Phase II of the project “Design and Implementation of 32 Bit RISC-V Processor with Parallel Graphics Accelerator”. A complete display system was designed in Verilog consisting of a RISC-V processor interface with instruction ROM, a memory-mapped GPU register block, a parallel graphics accelerator pixel datapath, a 320 × 240 × 12-bit block-RAM frame buffer, a pixel clock enable and a 640 × 480 VGA timing controller.

Behavioural simulation in Vivado confirmed the correct synchronisation pulses and colour outputs, and showed the processor reaching its idle loop at pc = 0x34 with instr = 0x0000006F. The design was then implemented and loaded on the Nexys A7-100T, and a sharp, stable, centred red rectangle of 320 × 240 pixels was displayed on a VGA monitor, in agreement with the simulation and with the design calculation of the rectangle position.

Power analysis showed that the total on-chip power of the current design is 2.675 W against 6.098 W for the earlier build, a reduction of 3.423 W (56.1 %), with I/O and logic power responsible for almost all of the saving and with the frame buffer and address arithmetic moved onto efficient block RAM and DSP resources. The junction temperature falls by 15.6 °C to 37.2 °C.

Future Scope

- Connect the processor program and the GPU register block to the frame-buffer write port so that the rectangle is drawn by software commands rather than by the demonstration writer.
- Replicate the accelerator lanes to generate several pixels per clock and measure the speed-up over software drawing.
- Add more graphics commands such as line, fill and sprite operations and a status register with busy and done flags.
- Repeat the power analysis on the fully implemented design with simulation-derived switching activity for higher confidence, and apply clock gating and I/O optimisation.
- Increase the resolution and colour depth, using an exact 25.175 MHz pixel clock from a clock-management block.

REFERENCES

[1] A. Waterman, Y. Lee, D. A. Patterson and K. Asanović, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.0," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-54, 2014.

[2] K. Asanović and D. A. Patterson, "Instruction Sets Should Be Free: The Case For RISC-V," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, 2014.

[3] K. Asanović et al., "The Rocket Chip Generator," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, 2016.

[4] P. D. Schiavone et al., "Slow and Steady Wins the Race? A Comparison of Ultra-Low-Power RISC-V Cores for Internet-of-Things Applications," in Proc. International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS), 2017.

[5] M. Gautschi et al., "Near-Threshold RISC-V Core With DSP Extensions for Scalable IoT Endpoint Devices," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 25, no. 10, pp. 2700–2713, 2017.

[6] C. Wolf, "PicoRV32 – A Size-Optimized RISC-V CPU," open-source project, 2015. [Online]. Available: https://github.com/YosysHQ/picorv32

[7] F. Elsabbagh et al., "Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics," in Proc. IEEE/ACM International Symposium on Microarchitecture (MICRO), 2021.

[8] K. Andryc, M. Merchant and R. Tessier, "FlexGrip: A Soft GPGPU for FPGAs," in Proc. International Conference on Field-Programmable Technology (FPT), 2013.

[9] R. Balasubramanian et al., "Enabling GPGPU Low-Level Hardware Explorations with MIAOW: An Open-Source RTL Implementation of a GPGPU," ACM Transactions on Architecture and Code Optimization, vol. 12, no. 2, 2015.

[10] J. Bush, P. Dexter, T. N. Miller and A. Carpenter, "Nyami: A Synthesizable GPU Architectural Model for General-Purpose and Graphics-Specific Workloads," in Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2015.

[11] J. D. Owens et al., "GPU Computing," Proceedings of the IEEE, vol. 96, no. 5, pp. 879–899, 2008.

[12] P. P. Chu, FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version. Hoboken, NJ, USA: Wiley, 2008.

[13] Video Electronics Standards Association, "VESA Monitor Timing Standard (DMT), Version 1.0, Revision 13," 2013.

[14] I. Kuon and J. Rose, "Measuring the Gap Between FPGAs and ASICs," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 26, no. 2, pp. 203–215, 2007.

[15] J. H. Anderson and F. N. Najm, "Power Estimation Techniques for FPGAs," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 12, no. 10, pp. 1015–1027, 2004.

[16] L. Shang, A. S. Kaviani and K. Bathala, “Dynamic Power Consumption in Virtex-II FPGA Family,” in Proc. ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA), 2002, pp. 157–164.

[17] A. P. Chandrakasan, S. Sheng and R. W. Brodersen, “Low-Power CMOS Digital Design,” IEEE Journal of Solid-State Circuits, vol. 27, no. 4, pp. 473–484, 1992.

[18] D. A. Patterson and J. L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[19] Digilent Inc., “Nexys A7 FPGA Board Reference Manual.”

[20] AMD (Xilinx), “Vivado Design Suite User Guide: Power Analysis and Optimization (UG907).”

[21] AMD (Xilinx), “7 Series FPGAs Memory Resources User Guide (UG473).”

[22] AMD (Xilinx), “Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics (DS181).”